

Filière : Master IESE – S2

Travaux Pratiques : Robotique Industrielle

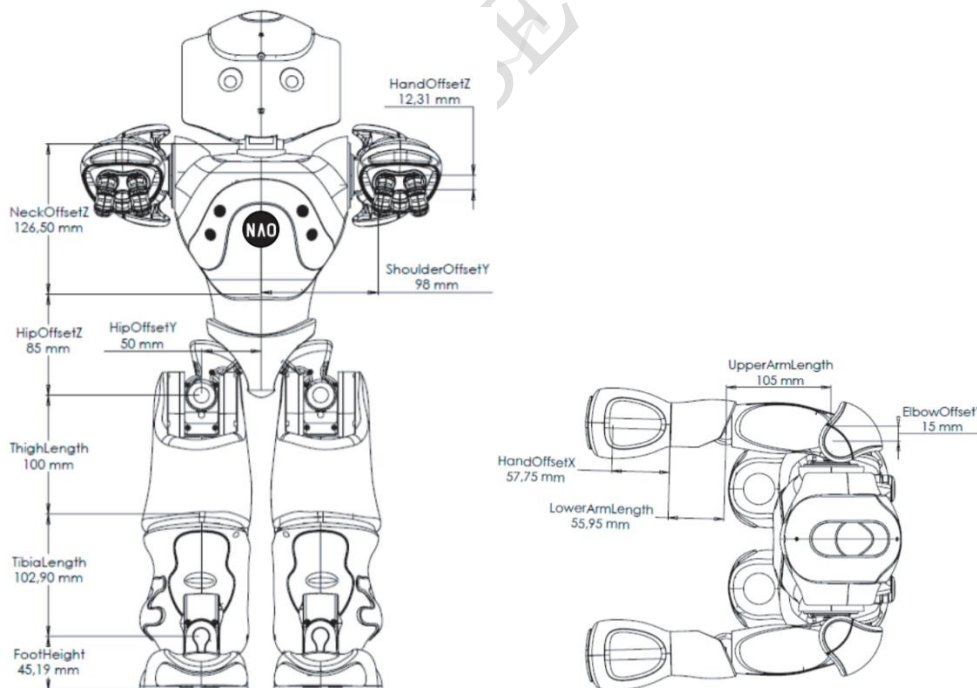
TP 3:

Programmation du Robot NAO6

Objectifs :

- Se familiariser avec la programmation de NAO en Python
- Comprendre l'utilisation de l'API NAOqi
- Apprendre à faire parler le robot, contrôler des mouvements, lectures des capteurs, réagir à des événements de base, ...etc

➔ Présentation du Robot NAO6 :



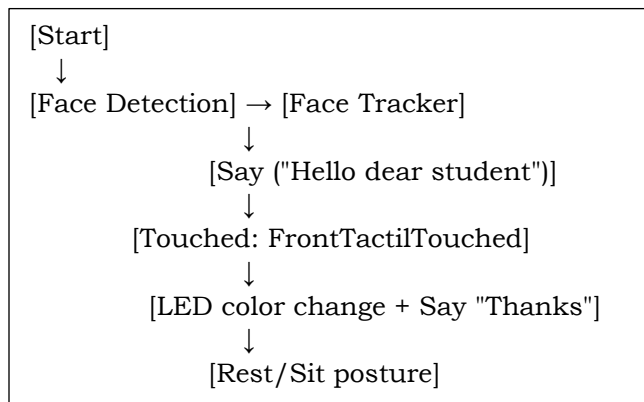


- **MiddleTactilTouched** → centre de la tête
- **RearTactilTouched** → arrière de la tête

→ Ajouter les blocs suivants :

Bloc	Fonction
Face Detection	Détecte un visage devant le robot
Face Tracker	Orienté la tête vers le visage détecté
Say	Fait parler NAO quand il détecte quelqu'un
Touched (Front Tactil)	Réagit au toucher sur la tête
ALLeds (via Python box)	Change les LEDs du visage (ex: vert ou bleu)
Rest ou Sit	Mettre NAO au repos à la fin du scénario

→ Enchaînement logique (exemple) :



⇒ → Tester avec d'autres capteurs tactiles : *Middle* ou *Rear*

2. Reconnaissance vocal (Speech recognition)

2.1. Reconnaissance vocal simple

Faire en sorte que NAO réagisse quand on lui dit un mot précis, par exemple :
 "Hello" → il commence à suivre un visage.

Étapes :

1. Ajouter un **bloc "Speech Recognition"**



2. Dans ses **propriétés**, entrer la **liste de mots-clés**, exemple : ["Hi", "Hello", "Robot"]
3. Connecter ce bloc à une suite d'actions
4. Optionnel : afficher ce qu'il a reconnu avec un bloc **"Say"**

Exemple : [Speech Recognition] → [Say: "command received!"] → [Face Tracker]

2.2. Utilisation du bloc Dialog (ALDialog)

Créer un **échange vocal** : NAO pose une question, attend une réponse, puis agit.

Étapes :

1. Ajouter un **bloc "Dialog"**
2. Cliquer sur le bouton d'édition (à droite) pour ouvrir l'éditeur de scripts de dialogue .*top*
3. Ajouter un fichier *dialog_1.top* avec une structure simple :

```
topic: ~question()
language: frf
proposal: "Souhaitez-vous commencer le suivi de visage ?"
u: (oui) "Super, je commence !"
u: (non) "Très bien, j'attends."
```

4. Ajouter un bloc **"Start Dialog"** et relier la sortie vers le comportement

3. Ajouter un son ou une animation préenregistrée

☞ Pour le son :

1. Glisser un **bloc "Play Sound"**
2. Sélectionner un son préexistant ou importer un fichier .*wav*
3. Connecter après une action (par exemple, après la détection du toucher)

☞ Pour l'animation :

1. Glisser un **bloc "Timeline" ou "Play Animation"**
2. Choisir une animation dans la bibliothèque de NAO (*animations/Stand/Gestures*)
3. Connecter dans le flux logique

Partie 3 : Programmation du Robot NOA avec Python

Dans cette partie, vous allez programmer le robot NAO en **Python** (NAOqi) après l'avoir utilisé avec Choregraphe. Nous commençons par un mémo des fonctions fréquemment utilisées, puis nous implémentons des scénarios concrets : suivi de visage, conversation simple, et mouvements coordonnés.

1. Choix du Python :

Raison	Explication
Simplicité	Python a une courbe d'apprentissage douce
Support officiel	Les robots NAO (en particulier NAO6 avec NAOqi 2.x) disposent d'un SDK Python officiel.
Multi-plateforme	Les scripts Python peuvent s'exécuter localement ou à distance (depuis un PC vers le robot).
Exemples riches	De nombreux exemples et tutoriels communautaires sont disponibles en Python.
Intégration facile	Python permet d'interfacer facilement les modules de vision, parole, capteurs et mouvements avec peu de lignes de code.

2. Exemple de fonctions couramment utilisées :

9559 : Est le port TCP/IP utilisé par défaut pour se connecter au gestionnaire de services NAOqi (TextToSpeech, Motion, Leds, etc.) sur le robot NAO.
ALProxy(service_name, robot_ip, port)
Mouvement (ALMotion)
motion.setAngles(joint_name, angle, speed) - joint_name : Nom de l'articulation (ex : "HeadYaw" pour tourner la tête à gauche/droite, "HeadPitch" pour monter/descendre la tête) - Angle : en rad - Speed : entre 0 (lent) et 1 (rapide)

`motion.getAngles(joint_name, useSensors)`

- `useSensors` : **True** → valeur réelle lue sur le robot, **False** → valeur cible
- Retourne un tableau contenant l'angle de l'articulation

`motion.rest()`

- Met le robot en position de repos (désactive les moteurs en sécurité)

`motion.stiffnessInterpolation(joint_name, stiffness, duration)`

- Active ou désactive les moteurs d'une articulation ou de tout le corps
- `stiffness` : de 0.0 (relâché) à 1.0 (rigide)
- `duration` : temps de transition (en secondes)

Postures (ALRobotPosture)

`posture.goToPosture(posture_name, speed)`

- `posture_name` : ex: **"StandInit"**, **"Crouch"**, **"Sit"**, **"LyingBack"**
- `speed` : entre 0 et 1

Synthèse vocale (ALTextToSpeech)

`tts.setParameter(parameter, value)`

- Modifie un paramètre vocal, par ex:
 - o **"pitchShift"** : hauteur de la voix (1.0 = normal)
 - o **"speed"** : vitesse de la parole

`tts.say("message")`

- Fait parler le robot avec le message donné

LED (ALLeds)

`leds.fadeRGB(group, color, duration)`

- `group` : nom du groupe de LEDs (ex: **"FaceLeds"**)
- `color` : code RGB en hexadécimal (ex: **0x00FF00** pour vert)
- `duration` : en secondes

`leds.off(group)`

- Éteint un groupe de LEDs

Capteurs et mémoire (ALMemory)

`memory.getData(key)`

- Récupère la valeur d'un événement ou capteur
- Exemple : **"FrontTactilTouched"**, **"HandRightBackTouched"**

Caméra (ALVideoDevice)

`video.subscribe(...) / video.getImageRemote(...)`

- Permet d'accéder aux images de la caméra (nécessite OpenCV pour traitement)

3. Prérequis et connexion au robot

☞ Déterminer l'adresse IP du robot, puis la tester avec un script simple.

Renseignez l'adresse IP du robot et créez les proxys aux services NAOqi :

```
from naoqi import ALProxy
import time
```



```
ROBOT_IP = "192.168.1.10" # @IP du Robot  
PORT = 9559
```

Services de base

```
tts = ALProxy("ALTextToSpeech", ROBOT_IP, PORT)  
motion = ALProxy("ALMotion", ROBOT_IP, PORT)  
memory = ALProxy("ALMemory", ROBOT_IP, PORT)  
posture = ALProxy("ALRobotPosture", ROBOT_IP, PORT)  
leds = ALProxy("ALLeds", ROBOT_IP, PORT)
```

(Optionnels selon les scénarios)

```
tracker = ALProxy("ALTracker", ROBOT_IP, PORT)  
asr = ALProxy("ASpeechRecognition", ROBOT_IP, PORT)  
behav = ALProxy("ALBehaviorManager", ROBOT_IP, PORT)
```

Exemple de fonctions :

```
from naoqi import ALProxy  
robot_ip = "192.168.1.10" # @IP du robot  
tts = ALProxy("ALTextToSpeech", robot_ip, 9559)  
tts.say("Bonjour, je suis NAO.")
```

☞ Contrôle des mouvements

```
motion = ALProxy("ALMotion", robot_ip, 9559)  
  
posture.goToPosture("StandInit", 0.5) # Changer la posture du robot  
motion.setAngles("HeadYaw", 1.0, 0.2) # Tourner la tête à droite  
time.sleep(2)  
motion.setAngles("HeadYaw", -1.0, 0.2) # Tourner la tête à gauche  
  
angle = motion.getAngles("HeadYaw", True) # Lire la position actuelle d'une articulation
```

☞ Changer la couleur des yeux du robot.

```
leds = ALProxy("ALLeds", robot_ip, 9559)  
leds.fadeRGB("FaceLeds", 0x00FF00, 1.0) # Vert
```

```
value = memory.getData("FrontTactilTouched") # Lire une donnée capteur ou événement  
face.subscribe("TestFace") # Activer la détection de visages  
tracker.track("Face") # Suivre automatiquement un visage
```

4. Scénarios

Scénario 1 – Suivi du visage

Objectif : le robot suit un visage avec la tête et allume ses yeux en vert lorsque le suivi est actif.

☞ **Étapes :**

- i. Activer le service **ALFaceDetection** ou **ALTracker**.
- ii. Mettre le robot en posture de départ et activer la rigidité.
- iii. Enregistrer la cible '**Face**' et démarrer le suivi.
- iv. Pendant le suivi, mettre les LEDs du visage en vert puis arrêter proprement.

Scénario 2 – Discussion simple (ASR + TTS)

Objectif : le robot écoute un mot-clé et répond. Langue : français/anglais.

☞ **Étapes :**

- i. Configurer la langue du module de reconnaissance vocale (ASR)
 - Définir la langue du robot sur *English*.
 - Choisir une liste de mots à reconnaître : par exemple *["Hello", "How are you", "Goodbye"]*.
 - Désactiver le *word spotting* (recherche libre) pour se limiter à ces mots-clés précis.
- ii. Souscrire au service **ASR** (Speech Recognition)
 - Créer une session d'écoute (donner un nom, par ex. *"ASR_Test"*).
 - Démarrer la reconnaissance et demander au robot de dire : *"I'm listening to you."*
- iii. Lire les mots reconnus via ALMemory
 - Récupérer les données associées à la clé *"WordRecognized"*.
 - Les résultats arrivent sous forme de liste : [mot, confiance, mot2, conf2, ...].
 - Vérifier si le niveau de confiance dépasse un seuil (ex. 0.4).
- iv. Réagir selon le mot reconnu
 - Si *hello* → répondre : *"Hello ! How can I help you?"*
 - Si *How are you* → répondre : *"I'm doing great, thank you!"*
 - Si *Goodbye* → répondre : *"Goodbye dear student !"* et arrêter l'écoute.
- v. Se désabonner du service ASR à la fin

- Libérer la ressource avec `asr.unsubscribe("ASR_Demo")`
- Utiliser `ALTextToSpeech` pour les réponses.
- Ajouter `ALSpeechRecognition` pour écouter une commande vocale simple ("Hello", "How are you ?").
- Extension : utiliser `ALDialog` pour poser des questions au lieu de simples mots-clés.

Scénario 3 – Mouvement coordonné

Objectif : exécuter une petite chorégraphie avec le bras droit et une phrase.

👉 Étapes :

- Mettre le robot en posture initiale stable
 - Utiliser la fonction `posture.goToPosture("StandInit", 0.6)` pour placer le robot debout.
 - S'assurer que le bras droit est actif : `motion.setStiffnesses("RArm", 1.0)`.
- Lever le bras droit
 - Définir les angles des articulations principales du bras droit (utiliser `setAngles`) :
 - `"RShoulderPitch"`
 - `"RShoulderRoll"`
 - `"RElbowYaw"`
 - `"RElbowRoll"`
 - `"RWristYaw"`
 - Appliquer une position levée (Ex. [-1.0, -0.2, 1.2, 0.9, 0.0]) avec une vitesse modérée (0.2).
 - Ajouter une pause pour que le mouvement se termine (Ex. 1 seconde).
- Faire le geste de salut
 - Faire bouger le coude ("RElbowRoll") d'avant en arrière plusieurs fois :
 - Passer de 0.4 à 0.9 radians à vitesse moyenne.
 - Répéter 3 fois avec des pauses de 0.5 seconde entre chaque mouvement.
- Faire parler le robot
 - Utiliser `tts.say("Hello dear students !")` pour synchroniser un message pendant le salut.
- Reposer le bras
 - Ramener le bras droit en position basse :



- Par exemple $[1.5, 0.0, 0.0, 0.0, 0.0]$ avec vitesse 0.2.
- Ajouter une pause pour stabiliser le mouvement.